

Draft Writeup: SPFC

Anonymous submission

Methodology

Comparison to be done against recent steering and compositional text-to-image generation methods:

- **Rectified CFG++** (NeurIPS 2025): a recent extension of classifier-free guidance for rectified flow models, focused on improving sampling fidelity and reducing guidance-related artifacts.
- **VSC** (ICCV 2025): a compositional generation method that decomposes a prompt into subprompts, generates intermediate visual representations, converts them into image tokens, and recombines them to improve final image generation.

Rectified CFG++ is the closest baseline for flow-based guidance, since it directly modifies the guidance behavior of rectified flow sampling. VSC is a strong compositional baseline, since it explicitly targets prompt decomposition and multi-concept generation.

For evaluation, we will use **T2I-CompBench** as the main benchmark because it directly tests compositional text-to-image alignment.

Rectified-Flow Sampling (base that is alr used)

Let a frozen text-to-image rectified flow predict velocities

$$v_{\theta}(x_t, t, c),$$

where x_t is the latent at noise time t and c is the text-conditioning embedding. The scheduler updates the latent by

$$x_{t_{n+1}} = \text{Step}(x_{t_n}, u_n, t_n),$$

where u_n is the velocity passed to the scheduler. For standard sampling,

$$u_n = v_{\theta}(x_{t_n}, t_n, c_P), \quad c_P = \text{TextEncode}(P).$$

All SPFC velocities use classifier-free guidance. With guidance scale s and null condition $c_{\emptyset}$,

$$\tilde{v}_{\theta}(x, t, c) = v_{\theta}(x, t, c_{\emptyset}) + s(v_{\theta}(x, t, c) - v_{\theta}(x, t, c_{\emptyset})).$$

To make explaining easy, we write

$$v(x, t, c) = \tilde{v}_{\theta}(x, t, c).$$

(also we don't modify how CFG works either, so for us we can also assume we are working on non CFG methods).

SPFC does not change anything. It only changes the velocity u_n supplied to the existing scheduler at selected timesteps (we select them currently manually).

Primitive Decomposition

Given a target prompt P , SPFC uses a source prompt S_0 and K primitive prompts $S_1, \dots, S_K$:

$$P \mapsto (S_0, S_1, \dots, S_K, P). \quad (1)$$

The source prompt preserves the main scene, medium, and style while removing difficult bindings. Each primitive is itself a complete prompt that isolates one semantic requirement of P , such as a count, color binding, material, spatial relation, reflection, or shadow. Complete primitive prompts are important: they make every condition propose a valid scene-level velocity rather than an ill-posed text fragment and MORE IMPORTANTLY, try to tie all of them to be close to the same distribution.

This step converts a hard compositional prompt into several simpler model queries. A target prompt may entangle object identity, attributes, counts, relations, and style in one conditioning vector. The primitives separate these requirements so the model can expose, through its own velocity field, which semantic components it currently knows how to move toward. The major assumption we make is that the model knows independently what each individual prompt semantically means, it just struggles to combine them, which we aim to fix.

Example:

- P : "A marble statue of a fox wearing green rain boots standing in front of a cracked mirror, with its shadow forming the shape of a dragon on the wall, cinematic lighting, realistic photo, sharp focus"
- S_0 : "A marble statue of a fox standing in front of a mirror, cinematic lighting, realistic photo, sharp focus"
- S_1 : "A marble statue of a fox standing in front of a wall, cinematic lighting, realistic photo, sharp focus"
- S_2 : "A fox statue wearing green rain boots, cinematic lighting, realistic photo, sharp focus"
- S_3 : "A marble fox statue reflected in a cracked mirror with multiple veins of cracks, cinematic lighting, realistic photo, sharp focus"

SPFC builds the ordered condition list

$$\mathcal{C} = (c_0, \dots, c_{K+1}), \quad c_i = \text{TextEncode}(Q_i), \quad (2)$$

where $Q_0 = S_0$, $Q_i = S_i$ for $1 \leq i \leq K$, and $Q_{K+1} = P$. Let

$$N = K + 2, \quad p = K + 1 \quad (3)$$

denote the number of conditions and the target index. Base priorities encode the intended roles:

$$w_i = \begin{cases} w_{\text{src}}, & i = 0, \\ w_{\text{prim},i}, & 1 \leq i \leq K, \\ w_{\text{tgt}}, & i = p, \end{cases} \quad (4)$$

with defaults $w_{\text{src}} = 0.7$, $w_{\text{prim},i} = 1.0$, and $w_{\text{tgt}} = 1.2$ (currently chosen arbitrarily, might change later after some more testing). The larger target base weight complements the target-centered correction and projection described below.

Sparse Intervention Schedule

Let T be the number of denoising steps and let

$$\mathcal{A} \subseteq \{0, \dots, T-1\}$$

denote the aggregation timesteps. $\mathcal{A}$ can be chosen however we want to choose them (preference to earlier timesteps).

For non-aggregation steps, SPFC matches target-only sampling:

$$n \notin \mathcal{A}: \quad u_n = v(x_{t_n}, t_n, c_P).$$

For aggregation steps, all condition velocities are evaluated at the same latent:

$$n \in \mathcal{A}: \quad v_i^{(n)} = v(x_{t_n}, t_n, c_i), \quad i = 0, \dots, N-1.$$

The target velocity is denoted $v_P^{(n)}$.

Velocity Agreement Gates

All gating signals are computed from the model’s own velocities at the current latent. For clarity, equations are written for one batch item; the implementation applies the same operations per sample. Define

$$\phi_i^{(n)} = \text{vec}(v_i^{(n)}). \quad (5)$$

The pairwise cosine agreement matrix is

$$M_{ij}^{(n)} = \frac{\langle \phi_i^{(n)}, \phi_j^{(n)} \rangle}{\|\phi_i^{(n)}\|_2 \|\phi_j^{(n)}\|_2 + \varepsilon}. \quad (6)$$

The consensus gate measures whether condition i agrees with the other condition velocities:

$$g_{i,\text{con}}^{(n)} = \text{clamp} \left(\frac{1}{N-1} \sum_{j \neq i} \max(0, M_{ij}^{(n)}), g_{\min}, g_{\max} \right). \quad (7)$$

The target-consistency gate measures whether condition i points in a direction compatible with the full prompt:

$$g_{i,\text{tgt}}^{(n)} = \text{clamp} \left(\max(0, M_{ip}^{(n)}), g_{\min}, g_{\max} \right). \quad (8)$$

Negative cosine components are discarded before clamping. Thus a primitive that is useful in isolation but points away from either the condition consensus or the target prompt receives little influence at that timestep.

$g_{\min}$ and $g_{\max}$ are the minimum and maximum gate values. They are currently set to $g_{\min} = 0$ and $g_{\max} = 1$ (arbitrary for now, might change after some more testing).

These gates are the method’s internal reliability test. Consensus discourages a single primitive from dominating when its velocity is an outlier relative to the rest of the scene. Target consistency prevents a primitive from improving its isolated concept by moving against the full prompt. The use of cosine agreement makes the test scale-invariant: a large primitive velocity is not trusted merely because its norm is large.

Target-Centered Velocity Composition

The raw score for each condition combines its role prior and the two data-dependent gates:

$$r_i^{(n)} = w_i g_{i,\text{con}}^{(n)} g_{i,\text{tgt}}^{(n)}. \quad (9)$$

The target score receives a numerical floor,

$$r_p^{(n)} \leftarrow \max(r_p^{(n)}, \varepsilon_r), \quad \varepsilon_r = 10^{-4}. \quad (10)$$

A temperature-scaled softmax gives a convex combination:

$$\alpha_i^{(n)} = \frac{\exp(r_i^{(n)}/\tau)}{\sum_{\ell=0}^{N-1} \exp(r_\ell^{(n)}/\tau)}. \quad (11)$$

Therefore $\alpha_i^{(n)} \geq 0$ and $\sum_i \alpha_i^{(n)} = 1$. The unclipped composed velocity is

$$v_{\text{raw}}^{(n)} = \sum_{i=0}^{N-1} \alpha_i^{(n)} v_i^{(n)}. \quad (12)$$

SPFC does not step directly with $v_{\text{raw}}^{(n)}$. Instead, it treats the composition as a correction to the target velocity:

$$\delta_v^{(n)} = v_{\text{raw}}^{(n)} - v_P^{(n)}. \quad (13)$$

For a reference vector a and ratio η , define norm clipping

$$\text{clip}_\eta(z; a) = z \min \left(1, \frac{\eta \|a\|_2}{\|z\|_2 + \varepsilon} \right). \quad (14)$$

With velocity clip ratio ρ , the Velocity Field Aggregation (VFA) output is

$$v_{\text{VFA}}^{(n)} = v_P^{(n)} + \text{clip}_\rho(\delta_v^{(n)}; v_P^{(n)}). \quad (15)$$

This yields the explicit stability bound

$$\|v_{\text{VFA}}^{(n)} - v_P^{(n)}\|_2 \leq \rho \|v_P^{(n)}\|_2. \quad (16)$$

The primitives can therefore change the target direction only within a fixed relative radius.

The softmax stage turns gate scores into a differentiable, convex selection over condition fields. The temperature τ controls how selective the method is: smaller values concentrate mass on the most trusted conditions, while larger values average more broadly. The subsequent target-centered clipping is crucial because a convex average of plausible prompt velocities can still be too far from the target trajectory. Clipping makes semantic steering subordinate to bounded deviation from P .

Latent Trajectory Projection

Velocity clipping bounds the proposed derivative. SPFC additionally bounds the scheduler-realized latent move. First compute the target and candidate next latents:

$$x_P^{+(n)} = \text{Step}(x_{t_n}, v_P^{(n)}, t_n), \quad (17)$$

$$x_{\text{VFA}}^{+(n)} = \text{Step}(x_{t_n}, v_{\text{VFA}}^{(n)}, t_n). \quad (18)$$

The candidate offset from the target trajectory and the target step length are

$$\Delta_x^{(n)} = x_{\text{VFA}}^{+(n)} - x_P^{+(n)}, \quad d_P^{(n)} = \left\| x_P^{+(n)} - x_{t_n} \right\|_2. \quad (19)$$

Using trust-region ratio r , define

$$\Pi_r(\Delta; d) = \Delta \min\left(1, \frac{rd}{\|\Delta\|_2 + \varepsilon}\right). \quad (20)$$

The aggregation-step update is

$$x_{t_{n+1}} = x_P^{+(n)} + \Pi_r(\Delta_x^{(n)}; d_P^{(n)}). \quad (21)$$

Consequently,

$$\left\| x_{t_{n+1}} - x_P^{+(n)} \right\|_2 \leq r \left\| x_P^{+(n)} - x_{t_n} \right\|_2. \quad (22)$$

The latent remains in a ball centered on the target-prompt scheduler step, with radius proportional to the target step size at that noise level.

Discussion

SPFC follows a very simple structure then. Run multiple flows, aggregate them on certain time steps, and repeat. The core idea remains: a diffusion model knows what you are talking about, you just need to explain it piece by piece and ask the model to tell you which pieces it understands and how to combine them.